Performance Analysis - Tweet Sentiment Pipeline

DSCC-402 Final Project

Zubaer Chowdhury

Overview

The pipeline follows a medallion flow with raw JSON tweets ingested into Bronze layer, mentions are cleaned and exploded in Silver layer, DistilBERT sentiment inference applied in Gold layer, and per-mention metrics are served through the Gold layer aggregation materialized application view. The overview of the Databricks pipeline metrics are given below:

Pipeline Layer Performance Metrics

Layer/Query	Measured runtime + rows	I/O and Spark data	Performance Comments
Bronze ingestion	19.159 s detailed run 42,275 rows written Stage list: 6.437 s.	3.21 MB written task time 36.75 s Photon 1%; no spill shown.	Mostly file ingestion/write overhead, not the bottleneck. Keep checkpointing stable and avoid unnecessary full refreshes.
Silver preprocessing	4.050 s detailed run 42,276 read -> 43,657 written. Stage list: 3.175 s	3.00 MB read, 5.02 MB written Batch Eval Python operator task time 1.97 s Photon 21%; spill 0 bytes	Regex UDF adds Python overhead, but current runtime is small. Replace Python UDF with native regex extraction to reduce serialization risk.
Gold ML inference	1 h 1 m 47 s detailed run 43,658 read -> 43,657 written Stage list incremental run: 2.068 s.	5.26 MB read, 5.28 MB written Arrow Eval Python operator; task time 1.03 h Photon 0%	Main bottleneck. Runtime is dominated by Python/transformer inference rather than data size or shuffle. Batch inference and cache/load model once per executor.
Application aggregation	4.687 s detailed run 20,950 read -> 17,985 written. Stage list: 2.810 s.	829.04 KB read, 496.89 KB written; Shuffle + Grouping Aggregate on mention task time 1.05 s Photon 82%; disk spill 0 bytes.	Current aggregation is healthy with small data, no spill, and high Photon use. Keep watching skew as mentions grow.

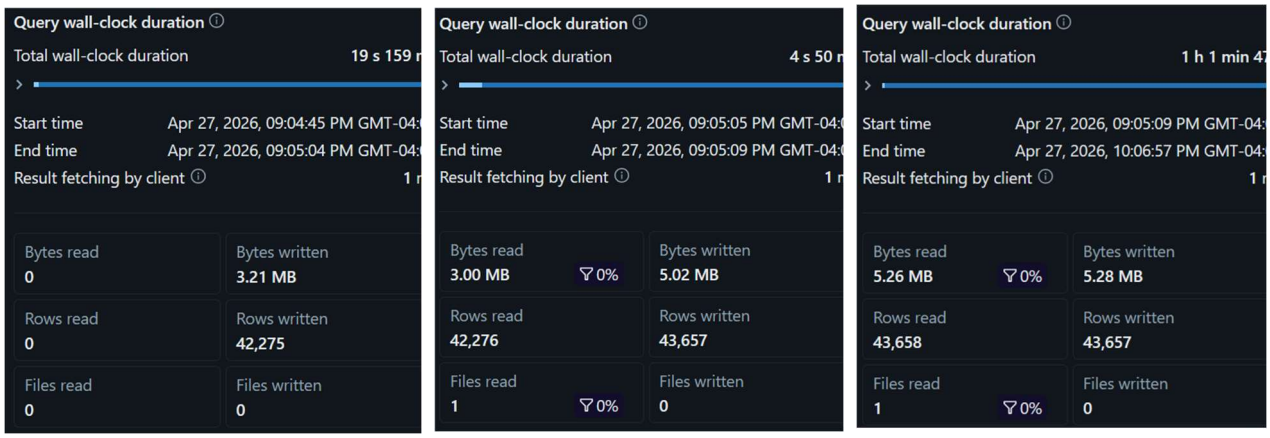

Fig. 1 Pipeline layer performance in Databricks

Optimizations Recommendations

Serialization / UDF overhead

The Gold layer is the clear bottleneck as 43.7K rows required 1.03 h of aggregate task time, with Photon at 0% and an Arrow Eval Python operator. This indicates model inference and Python execution dominate. The Silver layer also uses a Python regex UDF, but its runtime was only about four seconds. The recommended action is to use native Spark regex functions for mention extraction, and batch transformer inference by tuning Arrow batch size / model batch size while loading the model once per executor. This can make Silver layer overhead falls 30-50%, and Gold layer transformer throughput can improve several times over.

Shuffle, spill, and skew

The application view uses Shuffle and Grouping Aggregate on mention, but the measured run was small: 829 KB read, 17,985 rows written, 82% Photon task time, and 0 bytes spilled to disk. This means there is no current spill problem. However, mention data can become skewed because celebrity or brand handles receive many mentions. For better performance, keep Adaptive Query Execution enabled, coalesce small shuffle partitions, and use salting only if one or two handles begin creating long reducer tasks. This may prevent future straggler tasks while avoiding unnecessary complexity now.

Storage & Delta layout

The measured refresh read and wrote one file per layer, so file fragmentation was not visible in this run. Streaming pipelines often create small Delta files over repeated daily runs. Recommendation is to enable optimized writes/auto compaction for Silver and Gold layers after daily loads, optimize or liquid-cluster Gold layer by mention and timestamp because dashboard and aggregation workloads are mention/time-oriented. This can lower metadata planning overhead and faster dashboard scans as data accumulates.

Model Performance Analysis

Metric	Negative	Positive	Overall / Notes
Precision	0.682	0.759	Positive precision is stronger.
Recall	0.796	0.633	Positive recall is weaker; more positives are missed.
F1-score	0.735	0.690	Weighted F1 = 0.712
Support	21,707	21,950	Accuracy = 71.43% on 43,657 rows
Confusion matrix	TN 17,287 / FP 4,420	FN 8,052 / TP 13,898	Large FN count explains lower positive recall.

Conclusion

The pipeline is not constrained by shuffle or spill in the current 43K-row workload. The main issue is Gold-layer ML inference. The most effective optimization plan is to first batch/cache model inference, then replace Silver Python regex UDFs with native Spark expressions and finally maintain Delta layout using compaction or clustering as daily files accumulate. Model accuracy is 71.43%, with stronger negative recall than positive recall, so model-quality improvements should focus on preprocessing and reducing false negatives rather than changing the aggregation logic.